

Основы работы с контекстом в JavaScript

Пусть у нас есть какая-то функция `func`, внутри которой используется `this`:

```
1 function func() {  
2   console.log(this.value);  
3 }
```

На что указывает `this` в этой функции? А мы не знаем. И JavaScript не знает. И сама функция не знает. То есть в момент создания функции то, на что именно указывает `this`, не определено. И определится это только тогда, когда эта функция будет вызвана.

Пусть у нас есть какой-то инпут:

```
1 <input id="elem" value="text">
```

Привяжем к этому инпуту нашу функцию `func` так, чтобы она выполнялась по потере фокуса инпутом. Теперь в момент выполнения функции `this` будет указывать на наш инпут:

```
1 let elem = document.querySelector('#elem');  
2  
3 elem.addEventListener('blur', func);  
4 function func() {  
5   console.log(this.value); // по потере фокуса выведет 'text'  
6 }
```

Но ведь у нас может быть не один инпут, а несколько:

```
1 <input id="elem1" value="text1">  
2 <input id="elem2" value="text2">
```

И к каждому из этих инпутов мы можем привязать нашу функцию `func`. В этом случае для первого элемента `this` в момент вызова функции будет указывать на него, а для второго - на него.

На практике это значит, что `this` внутри функции зависит от того, на каком из инпутов мы потеряли фокус:

```
1 let elem1 = document.querySelector('#elem1');  
2 elem1.addEventListener('blur', func);  
3  
4 let elem2 = document.querySelector('#elem2');  
5 elem2.addEventListener('blur', func);  
6  
7 function func() {  
8   console.log(this.value); // выведет или 'text1', или 'text2'  
9 }
```

В общем-то, такая особенность `this` очень удобна - мы создаем всего одну функцию и привязываем ее к любому количеству инпутов. Если бы `this` не указывал на тот элемент, в котором произошло событие, то у нас бы такое не получилось - пришлось бы для каждого инпута создавать свою функцию с тем же кодом!

Контекст непривязанной функции в JavaScript

Давайте посмотрим, что будет, если в функции указать `this`, но не привязать ее ни к какому элементу, вот так:

```
1 function func() {  
2   console.log(this);  
3 }  
4  
5 func();
```

В этом случае результат зависит от того, строгий у нас режим или нет. Если режим нестрогий, то в `this` будет храниться ссылка на `window`:

```
1 function func() {
2   console.log(this); // ссылка на окно window
3 }
4
5 func();
```

А если режим строгий, то в `this` будет храниться `undefined`:

```
1 "use strict";
2
3 function func() {
4   console.log(this); // undefined
5 }
6
7 func();
```

Из сказанного выше может показаться, что в строгом режиме `this` всегда будет `undefined`. Как бы не так! Если просто вывести `this` вне функции, то в нем будет ссылка на `window` независимо от режима:

```
1 "use strict";
2 console.log(this); // в this ссылка на window
```

Выводы

В любой функции можно написать `this`, это не приведет к каким-либо ошибкам JavaScript. Но вот что именно будет лежать в этом `this` не определено до момента вызова функции. Причем при разных вызовах функции `this` может принимать разное значение. Все зависит от контекста, в котором была вызвана функция.

Потеря контекста в JavaScript

Пусть у нас есть функция в функции. Давайте внешнюю функцию назовем `parent`, а внутреннюю - `child`:

```
1 function parent() {
2   function child() {
3
4   }
5 }
```

Из предыдущих уроков вы знаете, что если задать какую-либо переменную во внешней функции - она будет доступна во внутренней:

```
1 function parent() {
2   let str = 'abcde';
3
4   function child() {
5     console.log(str); // выведет 'abcde'
6   }
7   child(); // вызываем внутреннюю функцию
8 }
9
10 parent(); // вызываем внешнюю функцию
```

Есть, однако, нюанс: внутренняя функция имеет доступ ко всем переменным внешней, но не имеет доступа `this`. То есть: если внешняя функция привязана к какому-то DOM элементу, то `this` в ней будет указывать на этот элемент, но `this` внутренней функции - не будет!

На что же тогда будет указывать `this` внутренней функции? Ответ: он будет равен `undefined` (в строгом режиме), так как функция ни к чему не привязана.

Давайте проверим на практике. Пусть у нас дан инпут:

```
1 | <input id="elem" value="text">
```

Привяжем к этому инпуту функцию `parent`, которая будет вызываться по потере фокуса инпутом:

```
1 | "use strict";
2 |
3 | let elem = document.querySelector('#elem');
4 | elem.addEventListener('blur', parent);
5 |
6 | // Вызовется по потере фокуса:
7 | function parent() {
8 |     // тут будет какой-то код
9 |
10 |     function child() {
11 |         // тут будет какой-то код
12 |     }
13 |     child(); // вызываем дочернюю функцию
14 | }
```

Давайте выведем содержимое `this` в консоль в двух местах: внутри функции `parent` и внутри функции `child`:

```
1 | "use strict";
2 |
3 | let elem = document.querySelector('#elem');
4 | elem.addEventListener('blur', parent);
5 |
6 | function parent() {
7 |     console.log(this); // выведет ссылку на наш инпут
8 |
9 |     function child() {
10 |         console.log(this); // выведет undefined
11 |     }
12 |     child();
13 | }
```

Запустите этот код, потеряйте фокус инпуту и посмотрите в консоль - вы увидите, что первый `console.log` выведет в консоль ссылку на наш инпут, а второй - просто `undefined`. Такая ситуация, когда `this` неожиданным для нас образом указывает не на то, что нам нужно, называется *потеря контекста*.

№ 1

©jsPmCxCL

Пусть теперь в одной и во второй функции мы будем выводить `value` инпута. Определите, что выведется в строчках кода, отмеченных комментариями:

```
1 | "use strict";
2 |
3 | let elem = document.querySelector('#elem');
4 | elem.addEventListener('blur', parent);
5 |
6 | function parent() {
7 |     console.log(this.value); // что выведет?
8 |
9 |     function child() {
10 |         console.log(this.value); // что выведет?
11 |     }
12 |     child();
13 | }
```

Решение проблемы с контекстом в JavaScript

В предыдущем уроке вы увидели, как теряется контекст во внутренней функции. Давайте решим эту проблему. Существует несколько способов решения. Давайте их разберем в следующих уроках.

Контекст через переменную в JavaScript

Решение следующее: во внешней функции запишем `this` в любую переменную и эта переменная будет доступна во внутренней функции, как и все переменные (обычно эту переменную называют `self`). Таким образом мы передадим `this` из внешней функции во внутреннюю:

```
1 | "use strict";
2 |
3 | let elem = document.querySelector('#elem');
4 | elem.addEventListener('blur', parent);
5 |
6 | function parent() {
7 |   console.log(this.value); // выведет 'text'
8 |
9 |   let self = this; // запишем this в любую переменную, например, в self
10 |
11 |   function child() {
12 |     console.log(self.value); // выведет 'text'
13 |   }
14 |   child();
15 | }
```

№ 1

©jsPmCxVS

Пусть дан такой код:

```
1 | <input id="elem" value="3">
2 |
3 | "use strict";
4 | let elem = document.querySelector('#elem');
5 | elem.addEventListener('blur', func);
6 | function func() {
7 |   alert( square() );
8 |
9 |   function square() {
10 |     return this.value * this.value;
11 |   }
12 | }
```

Автор кода хотел, чтобы по потере фокуса на экран вывелся квадрат числа из `value` инпута. Однако, почему-то при потере фокуса в консоль выдается ошибка. Исправьте ошибку автора кода. Напишите текст, в котором вы дадите объяснение автору кода, почему возникла его ошибка.

Контекст через параметр в JavaScript

Существует и другое решение проблемы. Сделаем так, чтобы функция `child` принимала параметр:

```
1 | function child(param) {
2 |   // тут будет код
3 | }
```

А при вызове этой функции будем передавать в нее `this`:

```

1 function parent() {
2   child(this); // передаем параметром this
3
4   function child(param) {
5     // в переменной param лежит переданное содержимое this
6   }
7 }

```

Так как вызов `child` осуществляется в родительской функции, то и передаваемый `this` указывает на то, что нужно. Затем этот `this` попадает в параметр `param` и в таком виде и будет доступен внутри функции.

Вот окончательный код:

```

1 let elem = document.querySelector('#elem');
2 elem.addEventListener('blur', parent);
3
4 function parent() {
5   child(this); // передаем параметром this
6
7   function child(param) {
8     console.log(param.value); // выводим value инпута
9   }
10 }

```

№ 1

©jsPmCхPPS

Возьмите код из предыдущей задачи и исправьте проблему кода с помощью изученного второго способа.

Контекст через стрелочную функцию в JavaScript

Третье решение проблемы заключается в использовании [стрелочных функций](#), которые не так давно появились в JavaScript. Такие функции, помимо всего прочего, не имеют своего контекста, а сохраняют контекст родителя. Исправим нашу проблему с помощью стрелочной функции:

```

1 let elem = document.querySelector('#elem');
2 elem.addEventListener('blur', parent);
3
4 function parent() {
5   console.log(this.value); // выведет 'text'
6
7   let child = () => {
8     console.log(this.value); // выведет 'text'
9   }
10  child();
11 }

```

№ 1

©jsPmCхAFS

Возьмите код из предыдущей задачи и исправьте проблему кода с помощью изученного третьего способа.

Привязывание контекста через метод call в JavaScript

Итак, мы разобрали, как на самом деле работает `this`. Давайте теперь рассмотрим методы, которые позволяют принудительно указать, в каком контексте вызывается функция (то есть принудительно сказать, чему равен `this`).

Первый метод, который мы разберем, называется `call`. Давайте посмотрим на его работу на примере. Пусть у нас есть инпут:


```
1 | <input id="elem" value="text">
```

Давайте получим ссылку на этот инпут и запишем ее в переменную elem:

```
1 | let elem = document.querySelector('#elem');
```

Давайте теперь сделаем функцию func, внутри которой выведем this.value:

```
1 | function func() {  
2 |   console.log(this.value);  
3 | }
```

Пока наша функция не знает, на что ссылается this. Вот, если бы мы ее привязали через addEventListener, тогда да. Но мы не будем этого делать. В замен мы просто вызовем нашу функцию, сказав ей, что this должен быть равен elem.

Это делается вот так: func.call(elem). Этот код эквивалентен простому вызову функции func вот так: func(), только с условием, что this равен elem.

Итак, синтаксис метода call такой: функция.call(объект). Давайте соберем все вместе:

```
1 | let elem = document.querySelector('#elem');  
2 |  
3 | function func() {  
4 |   console.log(this.value); // выведет value инпута  
5 | }  
6 | func.call(elem);
```

№ 1

©jsPmCxCM

Дана функция:

```
1 | function func() {  
2 |   console.log(this.value);  
3 | }
```

Даны три инпута:

```
1 | <input id="elem1" value="text1">  
2 | <input id="elem2" value="text2">  
3 | <input id="elem3" value="text3">
```

С помощью метода call и функции func выведите на экран value каждого из инпутов.

Метод call с параметрами в JavaScript

Пусть теперь функция func принимает некоторые параметры, назовем их param1 и param2:

```
1 | function func(param1, param2) {  
2 |   console.log(this.value + param1 + param2);  
3 | }
```

При вызове функции через call можно передать эти параметры вот так:

```
1 | func.call(elem, param1, param2);
```

Пусть дан следующий код:

```
1 | <input id="elem" value="hello">

1 | let elem = document.querySelector('#elem');
2 |
3 | function func(surname, name) {
4 |     console.log(this.value + ', ' + name + ' ' + surname);
5 | }
6 |
7 | func(); // тут должно вывести 'hello, John Smit'
```

Добавьте в последнюю строчку метод `call` так, чтобы на экран вывелось 'hello, John Smit'. Слово 'hello' должно взяться из `value` инпута, а 'John' и 'Smit' должны быть параметрами функциями.

Привязывание контекста через метод `apply` в JavaScript

Давайте теперь рассмотрим метод `apply` для привязывания контекста. Он работает практически так же, как и метод `call`. Разница заключается в том, что в `apply` параметры передаются в виде массива, а не перечисляются через запятую. В зависимости от задачи бывает удобен то один, то другой метод.

Вот пример передачи параметров в метод `apply`:

```
1 | func.apply(elem, [param1, param2]);
```



Пусть дан следующий код:

```
1 | <input id="elem" value="hello">

1 | let elem = document.querySelector('#elem');
2 |
3 | function func(surname, name) {
4 |     console.log(this.value + ', ' + surname + ' ' + name);
5 | }
6 |
7 | func(); // тут должно вывести 'hello, John Smit'
```

Добавьте в последнюю строчку метод `apply` так, чтобы на экран вывелось 'hello, John Smit'. Слово 'hello' должно взяться из `value` инпута, а 'John' и 'Smit' должны быть параметрами функциями.

Привязывание контекста через метод `bind` в JavaScript

Следующий метод `bind` позволяет навсегда привязать контекст к функции. Своим результатом этот метод возвращает новую функцию, внутри которой `this` будет иметь жестко заданное значение.

Давайте посмотрим на примере.

Пусть у нас есть инпут:

```
1 | <input id="elem" value="text">
```

Пусть ссылка на этот инпут записана в переменную `elem`:

```
1 | let elem = document.querySelector('#elem');
```

Пусть у нас также есть следующая функция `func`:

```
1 | function func(param1, param2) {
2 |     console.log(this.value + param1 + param2);
3 | }
```

Давайте с помощью `bind` сделаем новую функцию, которая будет копией функции `func`, но `this` в ней всегда будет равен `elem`:

```
1 | let newFunc = func.bind(elem);
```

Теперь в переменной `newFunc` лежит функция. Давайте вызовем ее, передав в первый параметр `'1'`, а во второй `'2'` (напоминаю, что в `elem` лежит инпут с `value`, равным `'text'`):

```
1 | newFunc('1', '2');
```

Давайте соберем все вместе:

```
1 | let elem = document.getElementById('elem');
2 |
3 | function func(param1, param2) {
4 |   console.log(this.value + param1 + param2);
5 | }
6 |
7 | let newFunc = func.bind(elem);
8 | newFunc('1', '2'); // выведет 'text12'
```

Не обязательно записывать результат работы `bind` в новую функцию `newFunc`, можно просто перезаписать `func`. После этого `func` будет такой же функцией, как и была, но с жестко связанным `this`:

```
1 | func = func.bind(elem);
```

№ 1

©jsPmCxBM

Пусть дан следующий код:

```
1 | <input id="elem" value="hello">

1 | let elem = document.getElementById('elem');
2 |
3 | function func(name, surname) {
4 |   console.log(this.value + ', ' + name + ' ' + surname);
5 | }
6 |
7 | // тут напишите конструкцию с bind()
8 |
9 | func('John', 'Smit'); // тут должно вывести 'hello, John Smit'
10 | func('Eric', 'Luis'); // тут должно вывести 'hello, Eric Luis'
```

Напишите в указанном месте конструкцию с методом `bind` так, чтобы `this` внутри функции `func` всегда указывал на инпут из переменной `elem`.